Patrones de comportamiento: cómo se reparten el trabajo los objetos

Los patrones de comportamiento

Los creacionales resolvieron quién escribe el *new*, y los estructurales cómo se arman objetos con objetos. Los de comportamiento se ocupan de lo que pasa en tiempo de ejecución, cómo se reparten las responsabilidades entre los objetos y cómo se comunican para completar una tarea. Es el grupo más grande del catálogo, con once patrones, y esta pre clase presenta cuatro.

Lo que estos patrones atacan es la parte del programa que más cambia, que no son los datos sino las reglas y las acciones. Quién tiene que enterarse cuando algo cambia, qué pasa cuando se aprieta un botón, qué puede hacer un documento según en qué etapa está, cómo se calcula un envío. Escritas adentro de la clase que las usa, esas reglas se vuelven cadenas de *if* que crecen con cada caso nuevo, y la clase se abre cada vez. La solución que los cuatro patrones comparten es sacar ese comportamiento a un objeto propio. El objeto que lo necesita guarda una referencia a ese objeto, o una lista, y le delega.

Con esa forma común, lo que distingue a cada patrón es qué se convierte en objeto y quién lo maneja. En Observer, el objeto que cambia avisa a una lista de interesados, que entran y salen de la lista en tiempo de ejecución. En Command, lo que se convierte en objeto es un pedido, una acción, que se puede asignar a un botón, guardar en un historial y deshacer. En State, el contexto cambia el objeto por su cuenta, a medida que pasa por las etapas de su ciclo de vida. En Strategy, el objeto lo elige quien usa al contexto, para configurarlo. Observer ya se usó en la clase 12, así que acá se repasa con un ejemplo corto. Command, State y Strategy son nuevos, y los dos últimos se parecen tanto en el código que la pre clase termina con la diferencia entre ellos. La referencia más accesible sigue siendo Refactoring Guru (https://refactoring.guru/es/design-patterns), con los once del grupo.

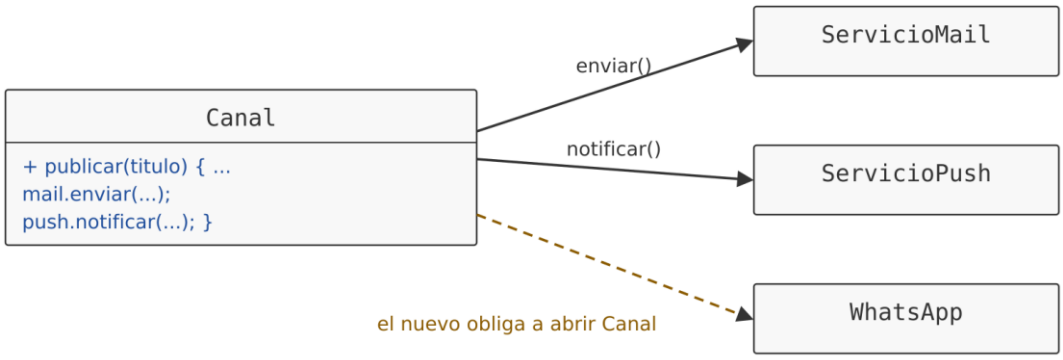
Los de comportamiento sacan una regla o una acción a un objeto propio, y el contexto le delega. Lo que cambia entre patrones es qué se saca y quién lo maneja.

Observer

El problema

Un canal publica videos y sus suscriptores quieren enterarse. Una persona quiere un mail, otra una notificación en el celular. La solución directa es que el canal, al publicar, llame al servicio de mail y al de notificaciones.

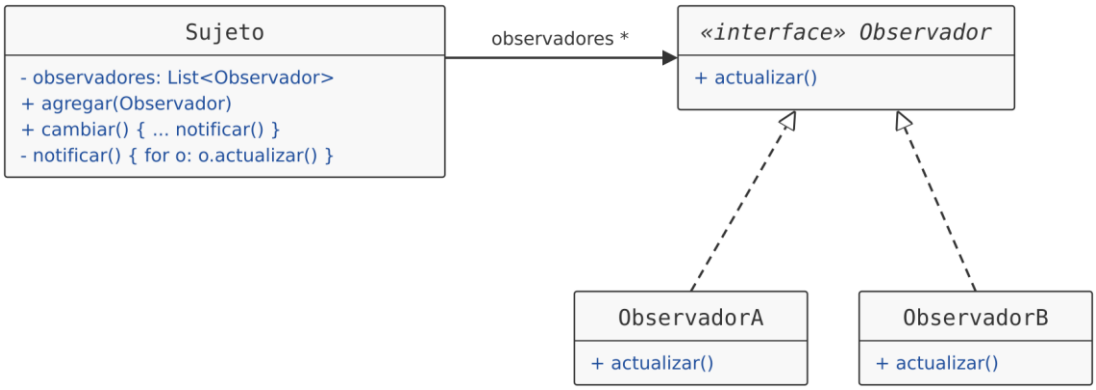
El costo es que el canal conoce cada forma de avisar, con sus clases y sus métodos, y una nueva, un mensaje de WhatsApp, obliga a abrirlo. Además la lista es fija, escrita en el código: nadie se puede suscribir ni desuscribir mientras el programa corre.



Sin el patrón. El canal conoce a cada forma de avisar por su clase y su método, y cada una nueva obliga a abrirlo.

La solución

El objeto que cambia, el Sujeto, define una interfaz con lo único que necesita de un interesado, un método para avisarle. Guarda una lista de objetos que implementan esa interfaz, ofrece métodos para entrar y salir de la lista, y después de cada cambio la recorre y avisa a cada uno. Los interesados, los Observadores, implementan la interfaz y hacen lo que quieran con el aviso. El Sujeto conoce la interfaz y nada más.



Estructura de Observer. El Sujeto guarda una lista de Observadores y los llama después de cada cambio. Conoce la interfaz, no las clases.

Ejemplo

```
public interface Suscriptor {
    void nuevoVideo(String titulo);
}

public class Canal {

    private final List<Suscriptor> suscriptores = new ArrayList<>();
```

```
    public boolean suscribir(Suscriptor suscriptor) {
        if (suscriptor == null) {
            return false;
        }
        this.suscriptores.add(suscriptor);
        return true;
    }

    public boolean desuscribir(Suscriptor suscriptor) {
        return this.suscriptores.remove(suscriptor);
    }

    public void publicar(String titulo) {
        for (Suscriptor suscriptor : this.suscriptores) {
            suscriptor.nuevoVideo(titulo);
        }
    }
}
```

Cada forma de avisar es una clase que implementa la interfaz:

```
public class AvisoPorMail implements Suscriptor {

    private final String direccion;

    public AvisoPorMail(String direccion) {
        this.direccion = direccion;
    }

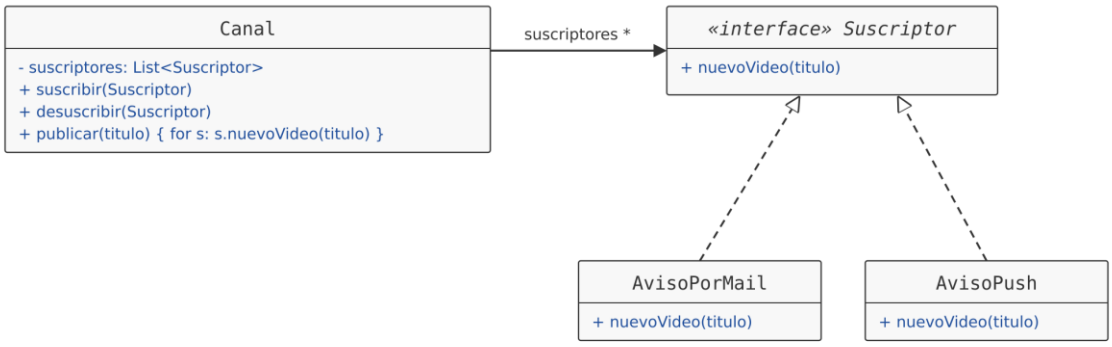
    @Override
    public void nuevoVideo(String titulo) {
        System.out.println("Mail a " + this.direccion + ": nuevo video, " + titulo);
    }
}
```

AvisoPush implementa la misma interfaz. Quien arma el sistema suscribe a los dos:

```
Canal canal = new Canal();
canal.suscribir(new AvisoPorMail("ana@correo.com"));
canal.suscribir(new AvisoPush());

canal.publicar("Patrones de comportamiento");    // llegan el mail y la notificacion
```

El *Canal* no importa a *AvisoPorMail* ni a *AvisoPush*. El aviso por WhatsApp es una clase que implementa *Suscriptor* y una línea al armar el sistema. Es la misma forma que tenía la *Caja* de la clase 12, con las ventanas como suscriptoras.



El ejemplo con la estructura de Observer. Canal avisa a lo que haya en su lista, y cada forma de avisar se suscribe al arrancar.

Ventajas y desventajas

- El Sujeto y los Observadores no se conocen por su clase. Un observador nuevo no toca al Sujeto, y los observadores entran y salen en tiempo de ejecución.
- Un cambio se comunica a cuantos objetos haga falta con un solo recorrido.
- El orden en que se avisa no está garantizado, y un observador que al recibir el aviso cambia otro sujeto puede desatar una cascada difícil de seguir.
- Un observador que dejó de usarse pero no salió de la lista sigue recibiendo avisos y sigue ocupando memoria, porque el Sujeto lo tiene referenciado. Hay que acordarse de desuscribirlo.

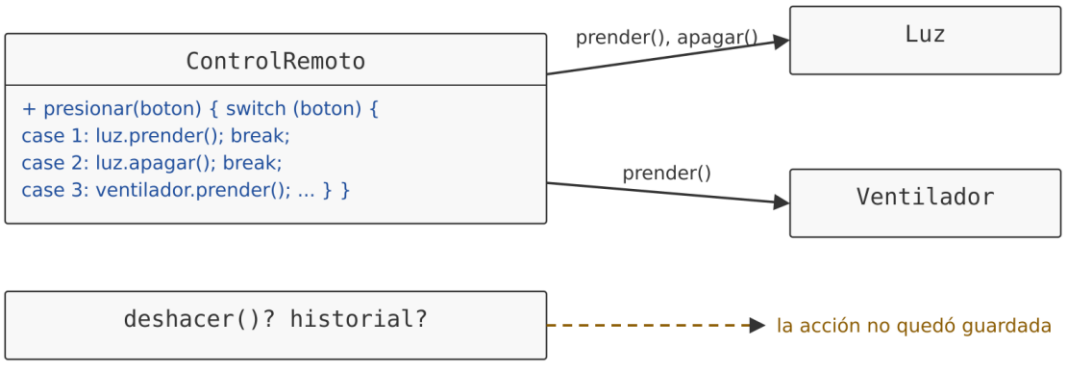
Observer. El sujeto define la interfaz del observador, guarda la lista y llama a cada uno después de cada cambio.

Command

El problema

Un control remoto de la casa tiene botones numerados, y cada uno prende o apaga algo. La solución directa es un método *presionar(boton)* con un *switch*: el 1 prende la luz del living, el 2 la apaga, el 3 prende el ventilador. El control conoce cada aparato y cada método, y cambiar qué hace un botón obliga a abrirlo.

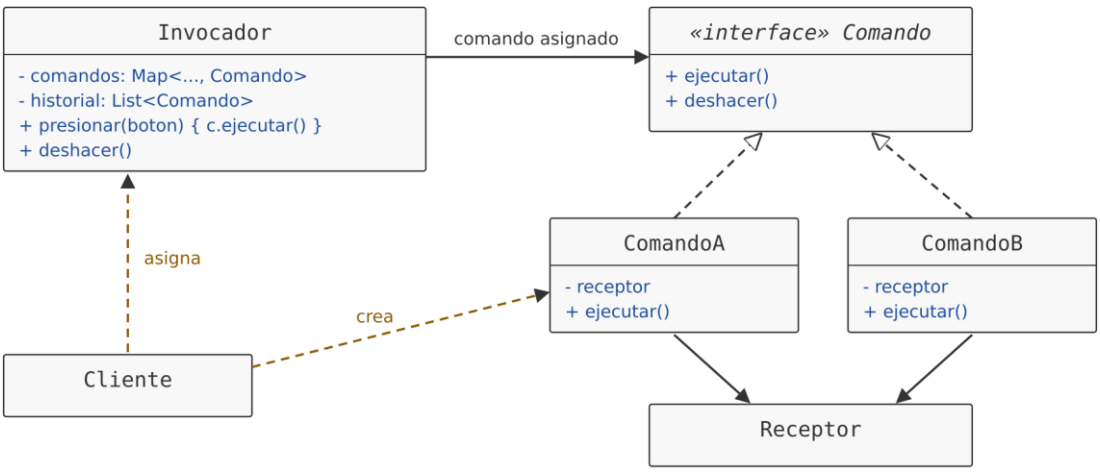
Hay un costo menos visible. La acción no existe como algo que se pueda guardar: el *switch* la hace y no queda nada. Si el control tiene que deshacer lo último, mostrar las últimas acciones o repetir una, no tiene con qué, porque lo único que recuerda es el número del botón.



Sin el patrón. El control conoce cada aparato y cada método, y la acción se hace en una rama del switch sin dejar rastro.

La solución

Cada pedido se convierte en un objeto. Se define una interfaz Comando con un método *ejecutar()*, y *deshacer()* si hace falta. Cada comando concreto guarda al objeto que hace el trabajo, el receptor, y sabe qué pedirle. El objeto que dispara las acciones, el invocador, guarda comandos, uno por botón, y cuando se aprieta uno llama a *ejecutar()* sin saber qué hace.



Estructura de Command. El Invocador ejecuta el Comando que tiene asignado sin saber qué hace, y el comando le pide el trabajo al Receptor.

Como el comando es un objeto, el invocador lo puede guardar en una lista después de ejecutarlo. Esa lista es el historial, y deshacer es sacar el último y llamar a su *deshacer()*.

Los participantes son cuatro. La interfaz Comando. Los comandos concretos, uno por acción, cada uno con su receptor. El Receptor, el objeto que sabe hacer el trabajo. Y el Invocador, que

guarda comandos y los ejecuta. Quien arma el sistema crea los comandos y se los asigna al invocador.

Es lo que hace *invokeLater()* desde la clase 10: recibe qué hacer, no cuándo. La lambda es un comando, y el hilo de eventos lo guarda en una cola y lo ejecuta después.

Ejemplo

Una luz que se prende y se apaga, y un control con botones y deshacer:

```
public interface Comando {
    void ejecutar();
    void deshacer();
}

public class PrenderLuz implements Comando {

    private final Luz luz;

    public PrenderLuz(Luz luz) {
        this.luz = luz;
    }

    @Override
    public void ejecutar() {
        this.luz.prender();
    }

    @Override
    public void deshacer() {
        this.luz.apagar();
    }
}
```

El control guarda un comando por botón, y una lista con los que ya ejecutó:

```
public class ControlRemoto {

    private final Map<Integer, Comando> botones = new HashMap<>();
    private final List<Comando> historial = new ArrayList<>();

    public boolean asignar(int boton, Comando comando) {
        if (comando == null) {
            return false;
        }
        this.botones.put(boton, comando);
        return true;
    }

    public boolean presionar(int boton) {
        Comando comando = this.botones.get(boton);
        if (comando == null) {
            return false;
        }
        comando.ejecutar();
        this.historial.add(comando);
        return true;
```

```
    }

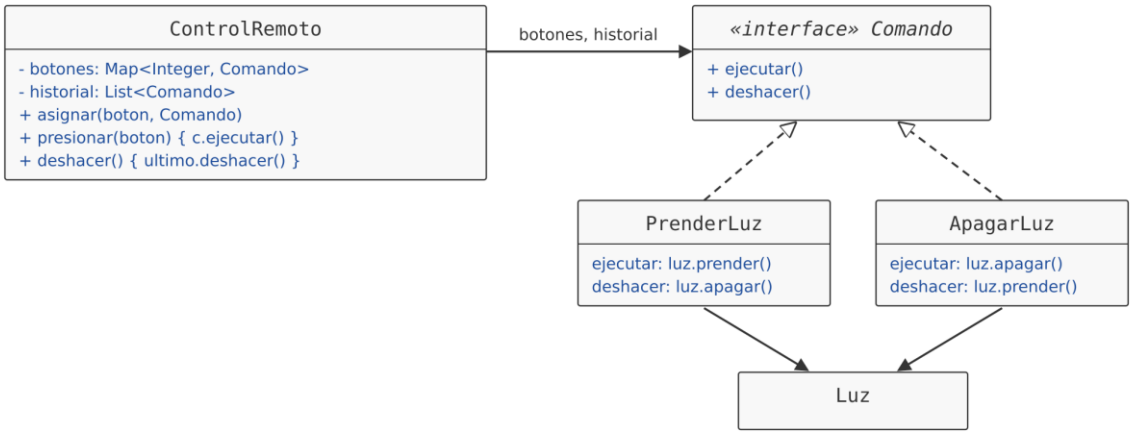
    public boolean deshacer() {
        if (this.historial.isEmpty()) {
            return false;
        }
        this.historial.remove(this.historial.size() - 1).deshacer();
        return true;
    }
}
```

Luz tiene *prender()* y *apagar()*, y *ApagarLuz* es *PrenderLuz* al revés. Quien arma el sistema asigna los botones:

```
Luz living = new Luz();
ControlRemoto control = new ControlRemoto();
control.asignar(1, new PrenderLuz(living));
control.asignar(2, new ApagarLuz(living));

control.presionar(1);    // la luz se prende
control.deshacer();      // y se vuelve a apagar
```

ControlRemoto no importa *Luz*. El ventilador es un receptor y dos comandos, y el control no se abre. Cambiar qué hace un botón es una línea en *asignar()*.



El ejemplo con la estructura de Command. ControlRemoto ejecuta y guarda comandos, y cada comando sabe qué pedirle a la Luz y cómo volver atrás.

Ventajas y desventajas

- Quien dispara la acción y quien la hace no se conocen. El mismo comando sirve para un botón, un ítem de menú y un atajo de teclado.
- Como el pedido es un objeto, se puede guardar: un historial, deshacer, una cola para ejecutar después, repetir el último.
- Es una clase por acción. Si no hace falta guardar nada, una lambda alcanza, y el patrón agrega clases sin ganar mucho.

- Deshacer no es gratis. Cada comando tiene que guardar lo necesario para volver atrás, y hay acciones que no se pueden revertir, como un mail enviado.

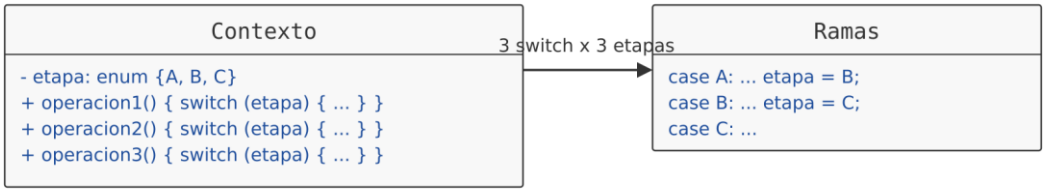
Command. El pedido se convierte en un objeto, el invocador lo ejecuta sin saber qué hace, y como es un objeto se puede guardar y deshacer.

State

El problema

Un objeto se comporta distinto según la etapa en la que está, y las etapas se recorren en un orden. Un documento es borrador, después está en revisión, después está publicado. En borrador se puede editar y enviar a revisión, en revisión se puede aprobar pero no editar, y publicado no se puede tocar. Un pedido, una partida, una conexión, un semáforo tienen la misma forma.

La solución directa es un atributo con la etapa actual, un *enum* o un entero, y un *switch* sobre ese atributo en cada método. Con tres etapas y tres operaciones son nueve ramas repartidas en tres *switch*. Una etapa nueva obliga a abrir todos los métodos, y las transiciones, qué etapa sigue a cuál, quedan desparramadas en asignaciones al atributo dentro de las ramas. Nadie puede leer en un solo lugar cómo funciona el ciclo de vida.

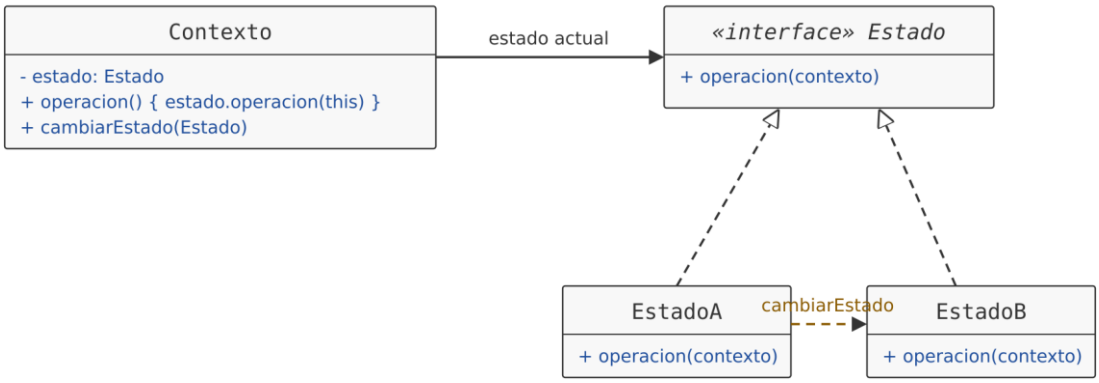


Sin el patrón. Un switch por método sobre el atributo de etapa, y las transiciones repartidas entre las ramas.

La solución

Cada etapa se convierte en una clase. Se define una interfaz Estado con las operaciones que dependen de la etapa, y una clase por etapa que la implementa. El objeto original, el Contexto, guarda una referencia al Estado actual y delega cada operación en él. La transición es cambiar esa referencia, y la puede hacer el propio Estado, que recibe el Contexto como parámetro para poder pedirle el cambio.

Los participantes son tres. El Contexto, con la referencia al estado actual y un método para cambiarla. La interfaz Estado. Y los estados concretos, uno por etapa, cada uno con el comportamiento de esa etapa y las transiciones que salen de ella.



Estructura de State. El Contexto delega en su Estado actual, y el Estado decide la transición cambiando la referencia del Contexto.

Desde afuera parece que el objeto cambia de clase. Quien lo usa llama siempre a los mismos métodos del Contexto, y lo que pasa depende de la etapa, sin que quien llama sepa cuál es.

Ejemplo

Un documento pasa por borrador, en revisión y publicado. Las operaciones son editar, enviar a revisión y publicar, y cada una devuelve si se pudo hacer en la etapa actual:

```
public interface EstadoDocumento {
    boolean editar(Documento documento, String texto);
    boolean enviarARevision(Documento documento);
    boolean publicar(Documento documento);
}

public class Documento {

    private EstadoDocumento estado = new Borrador();
    private String texto = "";

    public boolean editar(String texto) {
        return this.estado.editar(this, texto);
    }

    public boolean enviarARevision() {
        return this.estado.enviarARevision(this);
    }

    public boolean publicar() {
        return this.estado.publicar(this);
    }

    void cambiarEstado(EstadoDocumento nuevo) {    // lo usan los estados
        this.estado = nuevo;
    }

    void setTexto(String texto) {
        this.texto = texto;
```

```
    }
}
```

Documento no decide nada, le pasa cada operación a su estado. Cada etapa es una clase. En borrador se edita y se envía a revisión, pero no se publica:

```
public class Borrador implements EstadoDocumento {

    @Override
    public boolean editar(Documento documento, String texto) {
        documento.setTexto(texto);
        return true;
    }

    @Override
    public boolean enviarARevision(Documento documento) {
        documento.cambiarEstado(new EnRevision());
        return true;
    }

    @Override
    public boolean publicar(Documento documento) {
        return false;    // no se publica sin revisar
    }
}
```

En revisión ya no se edita, y publicar lleva a la última etapa:

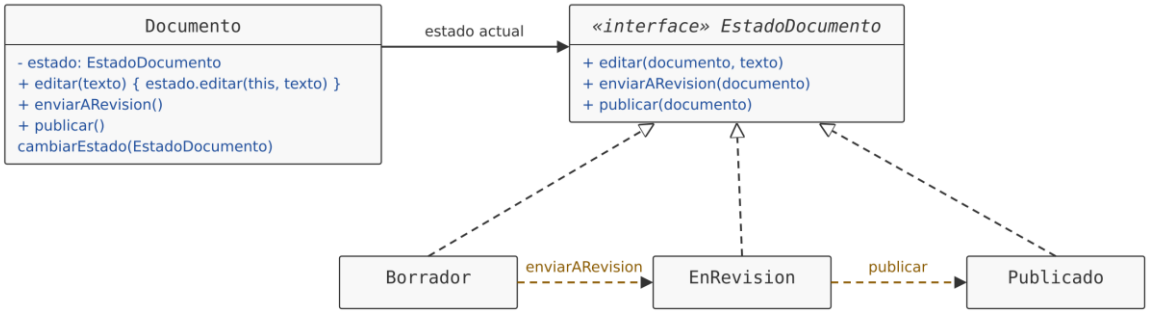
```
public class EnRevision implements EstadoDocumento {

    @Override
    public boolean editar(Documento documento, String texto) {
        return false;
    }

    @Override
    public boolean enviarARevision(Documento documento) {
        return false;    // ya esta en revision
    }

    @Override
    public boolean publicar(Documento documento) {
        documento.cambiarEstado(new Publicado());
        return true;
    }
}
```

Publicado devuelve *false* en las tres operaciones. Los métodos de *Documento* no tienen un solo *if*, y todo lo que se puede hacer en borrador está en *Borrador*. Una etapa nueva, “archivado”, es una clase más y un cambio en el estado desde el que se llega a ella.



El ejemplo con la estructura de State. Documento delega en su estado, y cada estado sabe qué permite y a cuál se pasa.

Ventajas y desventajas

- Todo el comportamiento de una etapa está en una clase, y las transiciones que salen de ella también. El ciclo de vida se lee clase por clase.
- El Contexto queda sin condicionales, y una etapa nueva es una clase nueva.
- Con muchas etapas son muchas clases, y si las transiciones las hacen los estados, cada estado conoce al siguiente, así que cambiar el orden del ciclo toca varias clases.
- Si el comportamiento cambia poco entre etapas, dos o tres *if* son más simples que el patrón.

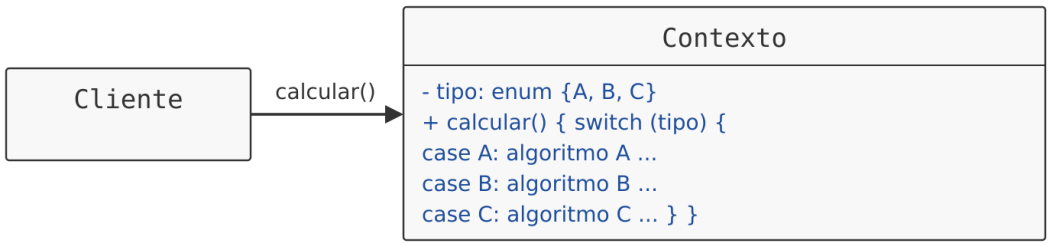
State. Una clase por etapa, el contexto delega en la actual, y la transición es cambiar la referencia.

Strategy

El problema

Una tarea se puede hacer de varias maneras, y cuál corresponde lo decide quien usa al objeto. El costo de envío de un pedido se calcula distinto para envío estándar, exprés o retiro en el local, y lo elige el cliente al comprar. Un archivo se puede comprimir con distintos algoritmos, una lista se puede ordenar por distintos criterios, un enemigo puede moverse con distintas tácticas.

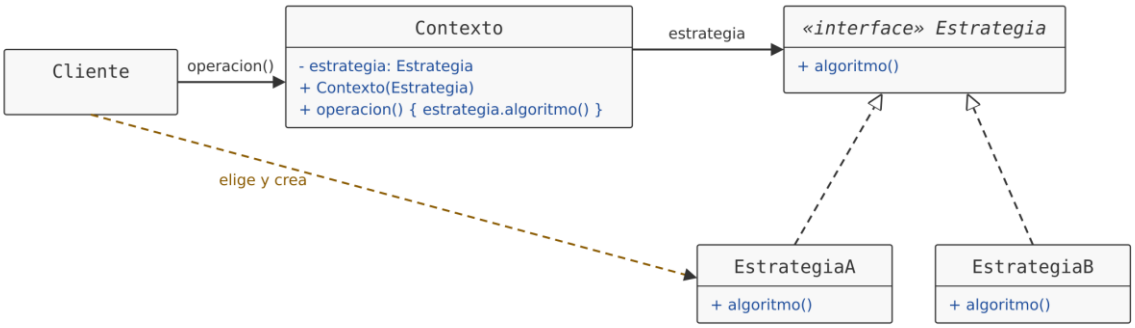
La solución directa es un método con un *switch* sobre el tipo, con cada algoritmo en una rama. La clase queda grande, mezcla su lógica con los detalles de cada algoritmo, y se abre con cada manera nueva. Y los algoritmos no se pueden reusar en otro lado, porque están adentro de las ramas.



Sin el patrón. Los algoritmos viven en las ramas de un switch dentro de la clase que los usa, y cada manera nueva la abre.

La solución

Cada manera de hacer la tarea se escribe en una clase propia, y todas implementan la misma interfaz, la Estrategia, con un único método que hace la tarea. El objeto que la necesita, el Contexto, guarda una Estrategia y le delega. Quien usa al Contexto le pasa la estrategia que quiere, por constructor o por un setter, y puede cambiarla después.



Estructura de Strategy. El Contexto delega en la Estrategia que le dieron, y el cliente elige cuál.

Los participantes son tres. La interfaz Estrategia. Las estrategias concretas, una por manera. Y el Contexto, que guarda una y la llama sin saber cuál es.

Ejemplo

Un pedido tiene peso y distancia de entrega, y el costo del envío depende de la modalidad que elige el cliente:

```
public interface EstrategiaEnvio {
    double costo(double pesoKg, int distanciaKm);
}

public class EnvioEstandar implements EstrategiaEnvio {

    @Override
```

```
    public double costo(double pesoKg, int distanciaKm) {
        return 500 + 120 * pesoKg;
    }
}

public class EnvioExpres implements EstrategiaEnvio {

    @Override
    public double costo(double pesoKg, int distanciaKm) {
        return 1500 + 200 * pesoKg + 8 * distanciaKm;
    }
}

public class RetiroEnLocal implements EstrategiaEnvio {

    @Override
    public double costo(double pesoKg, int distanciaKm) {
        return 0;
    }
}
```

Pedido guarda la estrategia que le dieron y le delega el cálculo:

```
public class Pedido {

    private final double pesoKg;
    private final int distanciaKm;
    private EstrategiaEnvio envio;

    public Pedido(double pesoKg, int distanciaKm, EstrategiaEnvio envio) {
        if (pesoKg <= 0 || distanciaKm < 0 || envio == null) {
            throw new IllegalArgumentException("Peso, distancia y envio son obligatorios");
        }
        this.pesoKg = pesoKg;
        this.distanciaKm = distanciaKm;
        this.envio = envio;
    }

    public boolean cambiarEnvio(EstrategiaEnvio envio) {
        if (envio == null) {
            return false;
        }
        this.envio = envio;
        return true;
    }

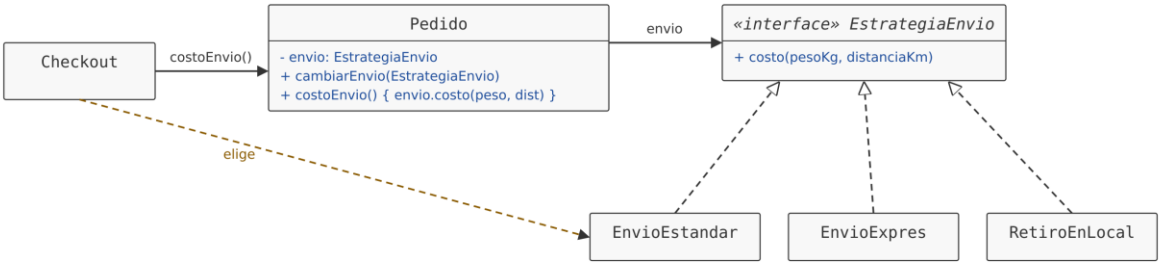
    public double costoEnvio() {
        return this.envio.costo(this.pesoKg, this.distanciaKm);
    }
}
```

El cliente elige al comprar y puede cambiar de idea antes de pagar:

```
Pedido pedido = new Pedido(2.5, 30, new EnvioEstandar());
pedido.costoEnvio();                        // 800

pedido.cambiarEnvio(new EnvioExpres());
pedido.costoEnvio();                        // 2240
```

Pedido no sabe cuántas modalidades hay ni cómo se calcula ninguna. Una modalidad nueva, envío en el día, es una clase con un método, y *Pedido* no se abre.



El ejemplo con la estructura de Strategy. Pedido delega el cálculo en la EstrategiaEnvio que le dio el cliente.

Ventajas y desventajas

- Los algoritmos están separados de la clase que los usa, así que se agregan, se cambian y se reusan sin abrirla.
- Se puede cambiar de algoritmo en tiempo de ejecución con una asignación.
- Quien usa al Contexto tiene que conocer las estrategias para elegir una. Con muchas, elegir se vuelve su propio problema.
- Si la estrategia necesita datos del Contexto, hay que pasárselos por parámetro, y la interfaz crece con parámetros que algunas estrategias no usan.

Strategy. Una clase por manera de hacer la tarea, el contexto delega en la que le dieron, y el cliente elige.

State y Strategy, la diferencia

Los dos diagramas son casi iguales. Un contexto, una interfaz y varias implementaciones, y el contexto delega. La diferencia no está en la estructura sino en quién cambia la referencia y por qué.

En Strategy la cambia el cliente, desde afuera, para configurar al contexto. Las estrategias no se conocen entre sí y el contexto no cambia de estrategia por su cuenta. Un *Pedido* con envío exprés lo va a tener hasta que alguien le dé otro. En State la cambia el propio objeto, desde adentro, porque pasó algo. Los estados se conocen, porque cada uno sabe cuál sigue, y quien usa al contexto no elige el estado ni suele saber cuál es. Un *Documento* pasa a revisión porque alguien lo envió, no porque alguien le asignó el estado.

La pregunta para decidir es una sola. Si el comportamiento lo elige quien usa al objeto, es Strategy. Si el objeto cambia de comportamiento solo, a medida que le pasan cosas, es State.

Misma estructura, distinta intención. Strategy es cómo hace algo el objeto, y lo elige el cliente. State es en qué etapa está, y cambia solo.

En la biblioteca

Observer es toda la interfaz gráfica. Un botón con *addActionListener()* es un sujeto con su lista, y *actionPerformed()* es el aviso. Command está en *javax.swing.Action*: una *AbstractAction* se le puede dar a un botón, a un ítem de menú y a un atajo de teclado, y los tres disparan la misma acción. Y cada *Runnable* que se le pasa a *invokeLater()* es un comando que espera en una cola. Strategy está en *Collections.sort(lista, comparador)*, donde el algoritmo de ordenar es de la biblioteca y el criterio de comparación, el *Comparator*, lo escribe quien llama. Un *LayoutManager* es una estrategia de un panel, y cambiar la disposición es cambiar el objeto. State casi no se ve desde afuera, porque es una decisión interna de la clase que lo usa. Un *Thread* se comporta distinto según esté nuevo, corriendo o terminado, y *start()* solo funciona una vez, pero esa máquina de estados está adentro.

Un botón es Observer, una Action es Command, un comparador es Strategy, y State es lo que hay adentro de un objeto que se comporta distinto según lo que le pasó.

Bibliografía sugerida para profundizar

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

- Capítulo 5, “Behavioral Patterns”, con “Command”, “Observer”, “State” y “Strategy”. El ejemplo de Command son los menús de un editor, con deshacer; el de State, una conexión de red; y el de Strategy, un editor que corta líneas de distintas maneras.

Freeman, E., & Robson, E. (2020). *Head First Design Patterns* (2.ª ed.). O’Reilly.

- Capítulo 1, “Welcome to Design Patterns”, que presenta Strategy con los patos; capítulo 2, “The Observer Pattern”; capítulo 6, “Encapsulating Invocation”, con Command y un control remoto; y capítulo 10, “The State of Things”, con la máquina de chicles.

Shvets, A. (2020). *Dive Into Design Patterns*. Refactoring.Guru. Versión web gratuita en https://refactoring.guru/es/design-patterns

- La sección “Patrones de comportamiento”, y en particular la comparación entre State y Strategy al final de cada uno.

Oracle. *The Java Tutorials*, “Object Ordering”, “How to Write an Action Listener” y “How to Use Actions”.

- El *Comparator* como estrategia de ordenamiento, el *ActionListener* como observador y la *Action* como comando, los usos de la biblioteca que aparecen en esta pre clase.